Module 7 : Les sous-programmes

Situations

Distance entre deux points

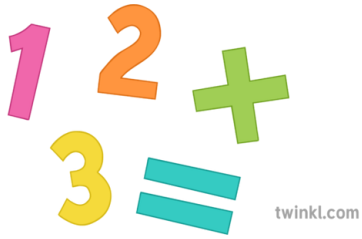

Figure 1, Distance entre deux points

Comparer les deux programmes suivants :

Version 1

```
# Saisie des coordonnées des points*
x1 = int(input("x1 > 0 ? "))
while x1 <= 0:
    x1 = int(input("x1 > 0 ? "))

y1 = int(input("y1 > 0 ? "))
while y1 <= 0:
    y1 = int(input("y1 > 0 ? "))

x2 = int(input("x2 > 0 ? "))
while x2 <= 0:
    x2 = int(input("x2 > 0 ? "))

y2 = int(input("y2 > 0 ? "))
while y2 <= 0:
    y2 = int(input("y2 > 0 ? "))

# calcul des distances
d1 = (x1 * x1 + y1 * y1)**0.5
d2 = (x2 * x2 + y2 * y2)**0.5

# affichage
if d1 < d2:
    print("P1 est plus proche de l'origine")
elif d1 > d2:
    print("P2 est plus proche de l'origine")
else:
    print("P1 et P2 sont équidistants")
```

Version 2

```
def saisie(msg):
    n = int(input(msg))
    while n <= 0:
        n = int(input(msg))
    return n

def dist(x, y):
    return (x * x + y * y)**0.5

def afficher(d1, d2):
    if d1 < d2:
        print("P1 est plus proche de l'origine")
    elif d1 > d2:
        print("P2 est plus proche de l'origine")
    else:
        print("P1 et P2 sont équidistants")


# PP
x1 = saisie("x1 ? ")
y1 = saisie("y1 ? ")
x2 = saisie("x2 ? ")
y2 = saisie("y2 ? ")

d1 = dist(x1, y1)
d2 = dist(x2, y2)

afficher(d1,d2)
```

Fonction des programmes

Les deux programmes permettent à l'utilisateur de saisir les coordonnées de deux points du plan, puis d'afficher le point le plus proche de l'origine.

Différences dans la structure du programme

Version 1	Version 2
Saisie Chaque saisie est effectuée à l'intérieur d'une boucle `while`, qui se répète tant que l'utilisateur ne saisit pas un nombre positif. Le même est répété 4 fois poru saisir les coordonnées des deux points.	**Saisie** La fonction `saisie()` gère la saisie d'un nombre positif. Les coordonnées des points sont saisies en appelant la fonction `saisie()`.
Calcul de la distance La distance `d1` et `d2` sont calculées en utilisant la formule de distance.	**Calcul de la distance** La fonction `dist()` est chargée de calculer la distance.
Affichage du point le plus proche Le point proche est déterminée à l'aide de la structure `if`.	**Affichage du point le plus proche** La fonction `afficher()` est chargée de déterminer le point le plus proche.

Résumé

La deuxième version est dite **modulaire**, car elle utilise des fonctions pour séparer les tâches de saisie, de calcul et d'affichage. Cela rend le code plus facile à comprendre et à maintenir. En outre, cette version :

- évite la répétition de code, car la saisie de chaque nombre est effectuée en appelant la même fonction.
- est plus élégante, car elle est mieux structurée que la première.
- peut sembler un peu plus complexe pour les débutants en programmation.

Algorithmes

Version 1

```
Algorithme Plus_Proche
Début
  Répéter
    Ecrire("x1 > 0 ? ") ; Lire(x1)
  Jusqu'à x1 > 0
  Répéter
    Ecrire("y1 > 0 ? ") ; Lire(y1)
  Jusqu'à y1 > 0
  Répéter
    Ecrire("x2 > 0 ? ") ; Lire(x2)
  Jusqu'à x2 > 0
  Répéter
    Ecrire("y2 > 0 ? ") ; Lire(y2)
  Jusqu'à y2 > 0

  d1 ← racine(x1 * x1 + y1 * y1)
  d2 ← racine(x2 * x2 + y2 * y2)

  Si d1 < d2 Alors
    Ecrire("P1 est plus proche de l'origine")
  Sinon Si d2 < d1 Alors
    Ecrire("P1 est plus proche de l'origine")
  Sinon
    Ecrire("P1 et P2 sont équidistants")
  Fin Si
Fin
```

Version 2

Programme Principal

```
Algorithme Plus_Proche_V2
Début
  saisie("x1 ? ", x1)
  saisie("y1 ? ", y1)
  saisie("x2 ? ", x2)
  saisie("y2 ? ", y2)

  d1 ← dist(x1, y1)
  d2 ← dist(x2, y2)

  afficher(d1, d2)
Fin
```

TDOG

Objet	Type
x1, y1, x2, y2, d1, d2	réel

Procédure Saisie()

```
Procédure Saisie(msg: chaine; @n: réel)
Début
  Répéter
    Ecrire(msg) ; Lire(n)
  Jusqu'à n > 0
Fin
```

TDO

Objet	Type/Nature
x1, x2, y1, y2, d1, d2	réel

Fonction dist()

```
Fonction dist(x, y: réel):réel
Début
  retourner racine(x * x + y * y)
Fin
```

Procédure afficher()

```
Procédure afficher(d1, d2: réel)
Début
  Si d1 < d2 Alors
    Ecrire("P1 est plus proche de l'origine")
  Sinon Si d2 < d1 Alors
    Ecrire("P1 est plus proche de l'origine")
  Sinon
    Ecrire("P1 et P2 sont équidistants")
  Fin Si
Fin
```

Problème de lessive

Najla, Douja et Zohra ont fait leurs lessives aujourd'hui. Or, **Najla** fait sa lessive tous les 3 jours, **Douja** tous les 4 jours et **Zohra** tous les 6 jours.

Figure 2, Lessive

Questions

1. Combien passera-t-il de temps avant que les trois femmes ne refassent leurs lessives le même jour ?
2. En supposant que :
   - **Najla** fait la lessive tous les `lf1` jours. Avec `lf1 > 0`
   - **Douja** fait la lessive tous les `lf2` jours. Avec `lf2 > 0`
   - **Zohra** fait la lessive tous les `lf3` jours. Avec `lf3 > 0`

   Déterminer quand les trois femmes referons leurs lessives le même jour ?
3. Ecrire l'algorithme d'un programme pour résoudre ce problème.

Solution

1. On pourra déterminer graphiquement le temps requis pour voir les trois femmes faire leurs lessives le même jour. Et ce en utilisant l'échelle temporelle suivante :

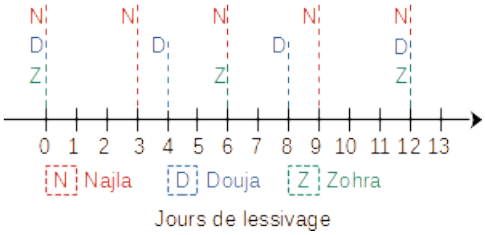

Figure 3, Jours de lessive

   On en déduit qu'il faudra attendre **12 jours**.
2. On remarque que le temps requis pour voir les trois femmes faire leurs lessives dans une même journée peut être calculé en utilisant la formule suivante :

   ppcm(3, 4, 6) = ppcm(3, ppcm(4, 6)) = ppcm(ppcm(3, 4), 6)= 12

   Plus généralement le **temps requis pour voir les femmes faire leurs lessives la même journée** est :

   ltf = ppcm(lf1, lf2, lf3) = ppcm(ppcm(lf1, lf2), lf3) = ppcm(lf1, ppcm(lf2, lf3))

   PPCM = Plus Petit Commun Multiple, c'est le plus petit nombre qui est multiple des trois nombres
3. L'algorithme de cette situation peut être écrit de deux façons :

1ère façon : méthode classique

```
Algorithme Lessive
Début
  // Saisie du jour de lessive de chaque femme
  Répéter
    Ecrire("Lessive femme 1 ? ") ; Lire(lf1)
  Jusqu'à (lf1 > 0)
  Répéter
    Ecrire("Lessive femme 2 ? ") ; Lire(lf2)
  Jusqu'à (lf2 > 0)
  Répéter
    Ecrire("Lessive femme 3 ? ") ; Lire(lf3)
  Jusqu'à (lf3 > 0)
  // Après combien de jours la 2ème et la 3ème femme font leurs lessives
  p1 ← lf2
  TantQue p1 mod lf3 ≠ 0 Faire
    p1 ← p1 + lf2
  Fin TantQue
  // Après combien de jours les 3 femmes font leurs lessives
  p2 ← lf1
  TantQue p2 mod p1 ≠ 0 Faire
    p2 ← p2 + lf1
  Fin TantQue
  ltf ← p2
  // Affichage du résultat
  Ecrire("Toutes les femmes feront leurs lessives après", ltf, "jours")
Fin
```

Objet	Type/Nature
lf1, lf2, lf3, ltf, p1, p2	entier

2ème façon : méthode modulaire

Programme Principal

```
Algorithme Lessive2
Début
  // Saisie du jour de lessive de chaque femme
  saisie("Lessive femme 1 ? ", lf1)
  saisie("Lessive femme 2 ? ", lf2)
  saisie("Lessive femme 3 ? ", lf3)
  // Après combien de jours les 3 femmes font leurs lessives
  ltf ← ppcm(lf1, ppcm(lf2, lf3))
  // Affichage du résultat
  Ecrire("Toutes les femmes feront leurs lessives après", ltf, "jours")
Fin
```

TDOG

Objet	Type/Nature
lf1, lf2, lf3, ltf	entier

Procédure saisie

```
procédure saisie(msg: chaîne, @nj: entier)
Début
  Répéter
    Ecrire(msg)
    Lire(nj)
  Jusqu'à (nj > 0)
Fin
```

TDOL

Objet	Type/Nature
-	-

Fonction ppcm

```
fonction ppcm(a, b: entier):entier
Début
  p ← a
  TantQue (p mod b ≠ 0) Faire
    p ← p + a
  Fin TantQue
  retourner p
Fin
```

TDOL

Objet	Type/Nature
p	entier

Fraction irréductible

En mathématiques, une **fraction est irréductible** s'il n'existe pas de fraction égale ayant des termes plus petits. Autrement dit, une fraction irréductible **ne peut pas être simplifiée**.

Théorème

Soient **a** un entier et **b** un entier naturel non nul. Alors $\frac{a}{b}$ est irréductible si et seulement si **a** et **b** sont premiers entre eux.

Exemple

La fraction $\frac{12}{20}$ n'est pas irréductible car 12 et 20 sont des multiples de 4 : $\frac{12}{20} = \frac{3 \times 4}{5 \times 4} = \frac{3}{5}$ (simplification par 4). On peut aussi écrire $\frac{12}{20} = \frac{12:4}{20:4} = \frac{3}{5}$.

La fraction $\frac{3}{5}$ est irréductible car 1 est le seul entier positif qui divise à la fois 3 et 5.

Méthode de simplification

Pour réduire directement une fraction, il suffit de **diviser le numérateur et le dénominateur par leur plus grand commun diviseur**. D'après le lemme de Gauss, cette forme réduite est unique.

Exemple

Pour réduire la fraction $\frac{42}{390}$, on calcule $\mathrm{PGCD}(42, 390) = 6$ puis on simplifie par 6 : $\frac{42}{390} = \frac{6 \times 7}{6 \times 65} = \frac{7}{65}$.

Problème

On souhaite écrire l'algorithme d'un **programme modulaire** qui calcule la somme de deux fractions :

$$\frac{p1}{q1}+\frac{p2}{q2}=\frac{p1\times q2+p2\times q1}{q1\times q2}=\frac{p3}{q3}=\frac{\frac{p3}{pgcd(p3,q3)}}{\frac{q4}{pgcd(p3,q3)}}=\frac{ps}{qs}$$

*Figure 4, Somme de deux fractions avec : p1, p2, ps ∈ ℤ et q1, q2, qs ∈ ℤ**

Solution

Solution non modulaire

```
Algorithme Somme_Fraction
Début
  // Saisie de p1 et q1
  Ecrire("Fraction : p1 / q1")
  Ecrire("Numérateur ? ") ; Lire(p1)
  Répéter
    Ecrire("Dénominateur ? ") ; Lire(q1)
  Jusqu'à q1 ≠ 0
  // Saisie de p2 et q2
  Ecrire("Fraction : p2 / q2")
  Ecrire("Numérateur ? ") ; Lire(p2)
  Répéter
    Ecrire("Dénominateur ? ") ; Lire(q2)
  Jusqu'à q2 ≠ 0
  // Calcul de la somme des deux fractions
  ps ← p1 * q2 + p2 * q1
  qs ← q1 * q2
  // Calcul du PGCD(ps, qs)
  a ← ps
  b ← qs
  TantQue b ≠ 0 Faire
    r ← a mod b
    a ← b
    b ← r
  Fin TantQue
  // Simplification
  ps ← ps div a
  qs ← qs div a
  // Affichage
  Ecrire(p1, "/", q1, "+", p2, "/", q2, "=", ps, "/", qs)
Fin
```

TDO

Objet	Type/Nature
p1, q1, p2, q2, ps, qs, a, b, r	entier

Solution modulaire

Programme Principal

```
Algorithme Somme_Fraction1
Début
  // Saisie de p1 et q1 / p2 et q2
  saisie_frac("Fraction : p1 / q1", p1, q1)
  saisie_frac("Fraction : p2 / q2", p2, q2)
  // Calcul de la somme des deux fractions
  somme_frac(p1, q1, p2, q2, ps, qs)
  // Simplification
  simp_frac(ps, qs)
  // Affichage
  Ecrire(p1, "/", q1, "+", p2, "/", q2, "=", ps, "/", qs)
Fin
```

TDOG

Objet	Type/Nature
p1, q1, p2, q2, ps, qs	entier

Procédure saisie_frac

```
Procédure saisie_frac(msg: chaîne, @p, @q: entier)
Début
  Ecrire(msg)
  Ecrire("Numérateur ? ") ; Lire(p)
  Répéter
    Ecrire("Dénominateur ? ") ; Lire(q)
  Jusqu'à q ≠ 0
Fin
```

TDOL

Objet	Type/Nature
-	-

Procédure somme_frac

```
Procédure somme_frac(a1, b1, a2, b2, @a3, @b3: entier)
Début
  a3 ← a1 * b2 + a2 * b1
  b3 ← b1 * b2
Fin
```

TDOL

Objet	Type/Nature
-	-

Procédure simp_frac

```
Procédure simp_frac(@a, @b: entier)
Début
  p ← pgcd(a, b)
  a ← a div p
  b ← b div p
Fin
```

TDOL

Objet	Type/Nature
p	entier

Fonction pgcd

```
Fonction pgcd(a, b: entier):entier
Début
  TantQue b ≠ 0 Faire
    r ← a mod b
    a ← b
    b ← r
  Fin TantQue
  retourner a
Fin
```

TDOL

Objet	Type/Nature
r	entier

Nombres amicaux

Deux nombres entiers naturels distincts sont dits « amicaux » (ou « amiables ») si la somme des diviseurs stricts de l'un est égale à l'autre et réciproquement.
Les diviseurs de 10 sont 1, 2, 5, 10. Les diviseurs stricts de 10 sont 1, 2, 5.

Travail demandé

1. Chercher la liste des diviseurs stricts de 220 et de 284.
2. Vérifier que 220 et 284 sont deux nombres amicaux.
3. Ecrire l'algorithme d'un programme qui vérifie si deux nombres données sont amicaux.

Solution

1. diviseurs(220) = {1, 2, 4, 5, 10, 11, 20, 22, 44, 55, 110}
   diviseurs(284) = {1, 2, 4, 71, 142}
2. som_div(220) = 1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284
   som_div(284) = 1 + 2 + 4 + 71 + 142 = 220
   ⇒ 220 et 284 sont des nombres amicaux.

3. **Solution non modulaire**

```
Algorithme amicaux
Début
  // Saisie de deux nombres strictement positifs
  Répéter
    Ecrire("Donner n ? ") ; Lire(n)
  Jusqu'à (n > 0)
  Répéter
    Ecrire("Donner m ? ") ; Lire(m)
  Jusqu'à (m > 0)
  // calcul de la somme diviseurs de n
  sn ← 0
  Pour i de 1 à n div 2 Faire
    Si n mod i = 0 Alors
      sn ← sn + i
    Fin Si
  Fin Pour
  // calcul de la somme diviseurs de m
  sm ← 0
  Pour i de 1 à m div 2 Faire
    Si m mod i = 0 Alors
      sm ← sm + i
    Fin Si
  Fin Pour
  // Affichage du résultat
  Si (n = sm) et (m = sn) Alors
    Ecrire(n, "et", m, "sont amicaux.")
  Sinon
    Ecrire(n, "et", m, "ne sont pas amicaux.")
  Fin Si
Fin
```

TDO

Objet	Type/Nature
n, m, i, sn, sm	entier

Travail demandé

Transformer la solution précédente en une solution modulaire.

Solution

Programme Principal

```
Algorithme amicaux
Début
  // Saisie de deux nombres strictement positifs
  saisie("Donner n ? ", n)
  saisie("Donner m ? ", m)
  // calcul de la somme diviseurs de n
  sn ← som_div(n)
  // calcul de la somme diviseurs de m
  sm ← som_div(m)
  // Affichage du résultat
  Si (n = sm) et (m = sn) Alors
    Ecrire(n, "et", m, "sont amicaux.")
  Sinon
    Ecrire(n, "et", m, "ne sont pas amicaux.")
  Fin Si
Fin
```

TDOG

Objet	Type/Nature
n, m, i, sn, sm	entier

Procédure saisie

```
Procédure saisie(msg: chaîne, @n: entier)
Début
  Répéter
    Ecrire("Donner n ? ") ; Lire(n)
  Jusqu'à (n > 0)
Fin
```

TDOL

Objet	Type/Nature
-	-

Fonction som_div

```
Fonction som_div(n: entier):entier
Début
  sn ← 0
  Pour i de 1 à n div 2 Faire
    Si n mod i = 0 Alors
      sn ← sn + i
    Fin Si
  Fin Pour
  retourner sn
Fin
```

TDOL

Objet	Type/Nature
sn, i	entier

Problème Inconnu

Soit l'algorithme suivant :

```
Algorithme Inconnu
Début
  // Séquence 1
  Répéter
    Ecrire("Donner la taille du tableau ? ") ; Lire(n)
  Jusqu'à (5 ≤ n ≤ 20)
  // Séquence 2
  Pour i de 0 à n-1 Faire
    Répéter
      Ecrire("Donner l'élément n°", i, " ? ") ; Lire(t[i])
    Jusqu'à ((i = 0) et (-10 ≤ t[i] ≤ 10)) ou ((i > 0) et (t[i] > t[i-1]))
  Fin Pour
  // Séquence 3
  Ecrire("Donner un entier ? ") ; Lire(a)
  // Séquence 4
  i ← -1
  Répéter
    i ← i + 1
  Jusqu'à (i = n) ou (t[i] > a)
  p ← i
  // Séquence 5
  Ecrire("p =", p)
Fin
```

Questions

1. Remplir le tableau de déclaration des objets (TDO) et le tableau de déclaration de nouveaux types (TDNT) relatifs à l'algorithme Inconnu.

TDO

Objet	Type
n, i, a, p	entier
t	tab

TDNT

tab = tableau de 20 entier

2. Donner le rôle des séquences 1, 2.
   Séquence 1 : permet de saisir la taille du tableau `n`.
   Séquence 2 : permet de remplir le tableau `t` par des valeurs strictement croissantes, le premier élément est dans l'intervalle [-10, 10].
3. Soit le tableau T suivant:

T	-2	1	5	7	12	26
	0	**1**	**2**	**3**	**4**	**5**

   a. Pour chaque valeur de `a`, donner la valeur de `p` correspondante après l'exécution des séquences 4 et 5 de l'algorithme.

Valeur de a	Valeur de p
-5	0
7	4
10	4
30	6

   b. Quel est le rôle de la variable `p` utilisée dans l'algorithme Inconnu ?
   La valeur de `p` indique l'indice de l'élément du tableau `t` dont la valeur est supérieure à `a`. Elle affiche `n` si a ≥ t[n-1].

4. Transformer cet algorithme en un algorithme modulaire, composé de procédures et de fonctions.

Programme Principal

```
Algorithme Inconnu_v2
Début
  Saisie(n)
  Remplir(n, t)
  Ecrire("Donner un entier ? ") ; Lire(a)
  p ← Position(a, n, t)
  Ecrire("p =", p)
Fin
```

TDOG

Objet	Type
n, a, p	entier
t	tab

Procédure Saisie

```
Procédure Saisie(@n: entier)
Début
  Répéter
    Ecrire("Donner la taille du tableau ? ") ; Lire(n)
  Jusqu'à (5 ≤ n ≤ 20)
Fin
```

TDOL

Objet	Type
-	-

Fonction Position

```
Fonction position(a, n:entier, t:tab):entier
Début
  i ← -1
  Répéter
    i ← i + 1
  Jusqu'à (i = n) ou (t[i] > a)
  retourner i
Fin
```

TDOL

Objet	Type
i	entier

5. Proposer l'Implémentation de votre algorithme en Python.

```python
from numpy import array

def saisie():
    n = 4
    while not (5 <= n <= 20):
        n = int(input("Donner la taille du tableau ? "))
    return n

def remplir(n):
    t = array([int()]*n)
    for i in range(n):
        t[i] = -11
        while not ((i == 0 and -10 <= t[i] <= 10) or (i > 0 and t[i] > t[i-1])):
            t[i] = int(input((f"Donner l'élément n°{i} ? ")))
    return t

def position(a, n, t):
    i = -1
    while True:
        i = i + 1
        if (i == n) or (t[i] > a):
            break
    return i
# PP
n = saisie()
t = remplir(n)
a = int(input("Donner un entier ? "))
p = position(a, n, t)
print("p =", p)
```

Notes de cours

Décomposition modulaire

L'**analyse modulaire**, appelée également **décomposition modulaire**, consiste à diviser un problème en sous problèmes de difficultés moindres.
En algorithmique, les **sous problèmes** correspondent à des **sous-programmes**.

Sous-programme

Un **sous-programme** est une section de code nommée ***qui peut être appelée*** en écrivant son nom dans une instruction du programme.
L'écriture de **sous-programmes** rend le code plus lisible et réutilisable, car le code est subdivisé en des sections plus petites.

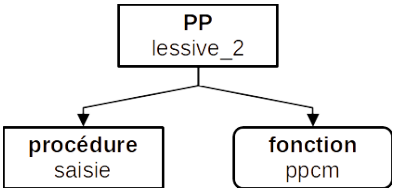

Figure 5, Décomposition modulaire du problème de lessive des trois femmes

En algorithmique on distingue deux catégories de **sous-progrmmes** : les **procédures** et les **fonctions**.
Une **procédure** exécute simplement un ensemble d'instructions, tandis qu'une **fonction** renvoie une valeur une fois son exécution est terminée.

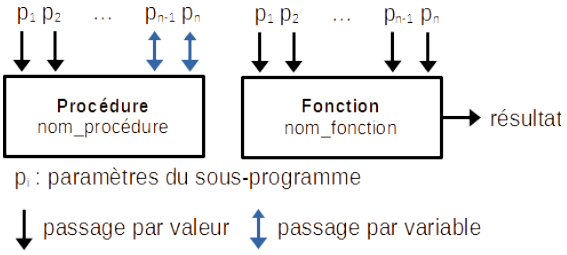

Figure 6, Différence entre une procédure et une fonction

La plupart des langages de programmation sont livrés avec un ensemble de sous-programmes intégrés (fonctions prédéfinies). Ils permettent, aussi, au programmeur d'écrire leurs propres sous-programmes personnalisés.

Fonction

Définition

Une **fonction** est un sous-programme **qui retourne à son appelant un seul résultat** *en fonction de ses paramètres*.
Une **fonction** peut avoir zéro ou plusieurs paramètres. Ces **paramètres** sont ***souvent transmis par valeur***.

Appel

Comme une **fonction renvoie toujours une valeur**, son appel peut se faire de différentes manières :

- Dans une affectation :

```
// pgcd(a, b) renvoie le PGCD des deux valeurs
dc ← pgcd(a, b)
```

- Dans une structure conditionnelle :

```
// Afficher si un nombre est premier
// premier(n) retourne vrai si n est un nombre premier
Si premier(n) Alors
  Ecrire(n, "est premier")
Sinon
  Ecrire(n, "n'est pas premier")
Fin Si
```

- Dans une structure itérative :

```
// f(x) est une fonction qui admet un extrémum
// en x0 ∈ [0, +∞[
// Recherche de l'extrémum de f(x)
x0 ← 0
TantQue (f(x0+pas) > f(x0)) Faire
  x0 ← x0 + pas
Fin TantQue
```

```
// Saisir une chaine alphabétique
// est_alphabetique(ch) : retrourne Vrai
// Si ch[i] ∈ ["A", "Z"] ∪ ["a", "z"]
Répéter
  Ecrire("Une chaine alphabétique ? ")
  Lire(ch)
Jusqu'à est_alphabetique(ch)
```

- Comme paramètre d'un autre sous-programme :

```
// somme_carre(a, b) renvoie a² + b²
Ecrire(somme_carre(a, b))
```

```
// calculer PGCD de a, b et c
dc ← pgcd(a, pgcd(b, c))
```

Vocabulaire et Syntaxe

Une fonction s'écrit comme suit en algorithmique :

```
Fonction NomFonction(p1: type1, p2: type2, ...,
                     pn: typen):typerésultat
  //
  // Traitements
  //
  Retourner résultat
Fin
```

Procédure

Définition

Une **procédure** est un sous-programme **qui ne retourne pas, directement, de résultats à son appelant**.
Une **procédure** peut avoir zéro ou plusieurs paramètres. Ces **paramètres** peuvent être, selon le besoin, ***transmis par valeur*** ou ***transmis par variable***.
En algorithmique, le **mode de passage par variable** *est utilisé pour renvoyer, indirectement, un ou plusieurs résultats à l'appelant*. Lorsqu'une procédure renvoie des résultats à travers ses paramètres, on dit qu'**elle possède un effet de bord**.

Appel

Comme une **procédure ne renvoie aucune valeur**, son appel se fait toujours de la même façon :

```
// Saisir une valeur dans n
saisir(n)
// Remplir le tableau t par n valeurs distinctes
remplir_tab(t, n)
// Echanger le contenu de deux variables
permuter(a, b)
```

Une **procédure** utilise les paramètres passés par valeur pour réaliser ses traitements. Elle peut, aussi, modifier la valeur des paramètres transmis par variable, directement, chez l'appelant.

Vocabulaire & Syntaxe

Une procédure s'écrit comme suit en algorithmique :

```
Procédure NomProcédure(p1: type1, p2: type2, ..., pn: typen)
  //
  // Traitements
  //
Fin
```

Paramètres et Mode de passage

Un **sous-programme** défini réellement un **ensemble de traitements effectués sur des données**. Ces **données** doivent être passées au sous-programme dans ***ses paramètres***.

Types de paramètres

On distingue deux types de paramètres :

- **Les paramètres formels :** utilisés lors de la définition d'un sous-programme.
- **Les paramètres effectifs :** utilisés lors de l'utilisation (l'appel) d'un sous-programme.

Dans la figure ci-dessous, les paramètres **a**, **b** et **c** qui figurent dans la définition du sous-programme **somme** sont appelés des paramètres **formels**
Les paramètres **x**, **y** et **z** utilisés dans le programme principal qui appelle le sous-programme **somme** sont dits **effectifs** ou **réels**.

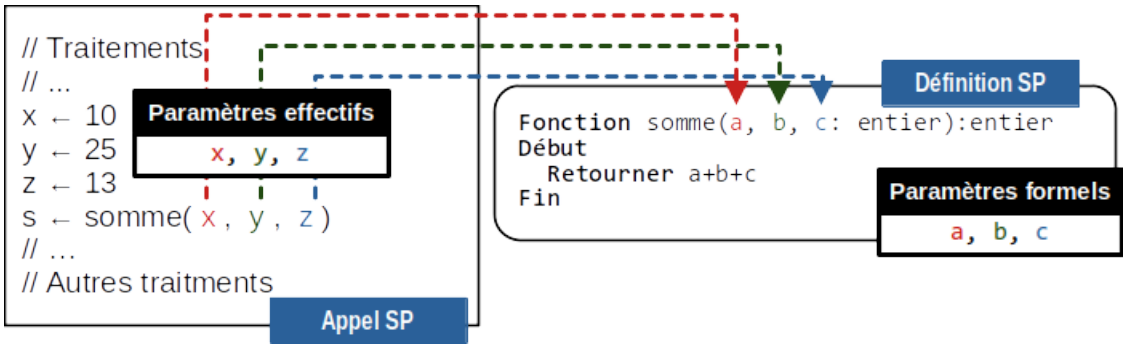

Figure 7, Les types de paramètres

Modes de passage des paramètres

Il existent deux modes de passage des paramètres : par variable ou par valeur.

Passage par valeur

Lors d'un **passage par valeur**, la valeur de l'expression passée en paramètre est copiée dans une variable locale. C'est cette variable qui est utilisée pour faire les traitements dans le sous-programme appelé.
Dans la figure suivante, la valeur de **y** est copiée dans le paramètre **x** de la fonction lors de l'appel de cette dernière.

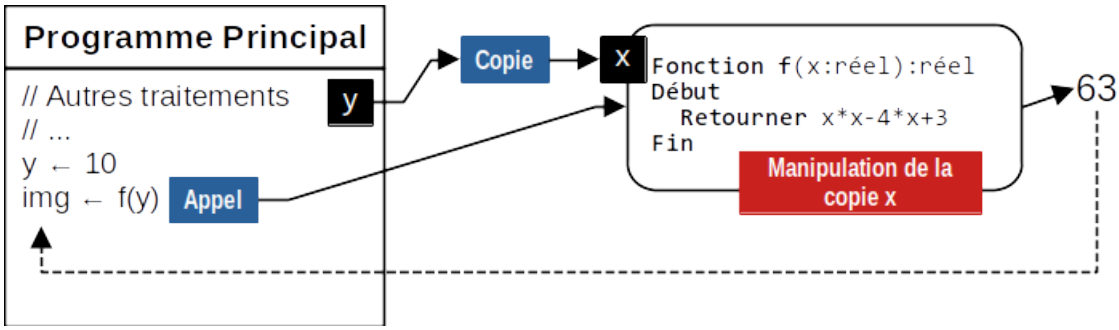

Figure 8, Passage par valeur

Passage par variable

Dans la figure suivante, l'adresse de la variable **y** est transmise à la procédure **saisie** sous le nom de **x**. Tout changement de la valeur de **x** dans la procédure à pour effet le changement de la valeur de **y**.
x et **y** sont deux noms différents de la même variable. La variable **y** dans le sous-programme est appelée **x**.

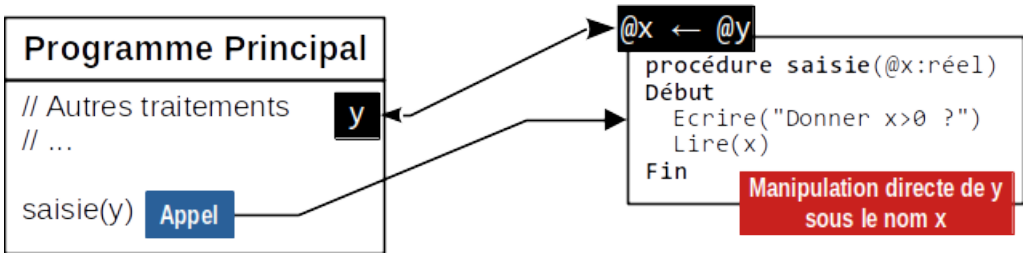

Figure 9, Passage par variable

Portée des variables

Toute **variable** a une durée de vie bornée **au bloc** où elle est déclarée. Ce bloc définit **la portée** de la variable.
Les **variables globales** sont déclarées dans le programme principal. Elles sont utilisables dans tout le programme.
Les **variables locales** sont déclarées dans un sous-programme. Elles ont une portée limitée et elles ne sont utilisables qu'à l'intérieur de celui-ci.
La fonction **existe** dans l'exemple suivant utilise les variables globales **t** et **n** qui sont déclarées dans le programme principal. **trouve** et **i** sont des variables locales car elles sont visibles uniquement à l'intérieur du sous-programme.

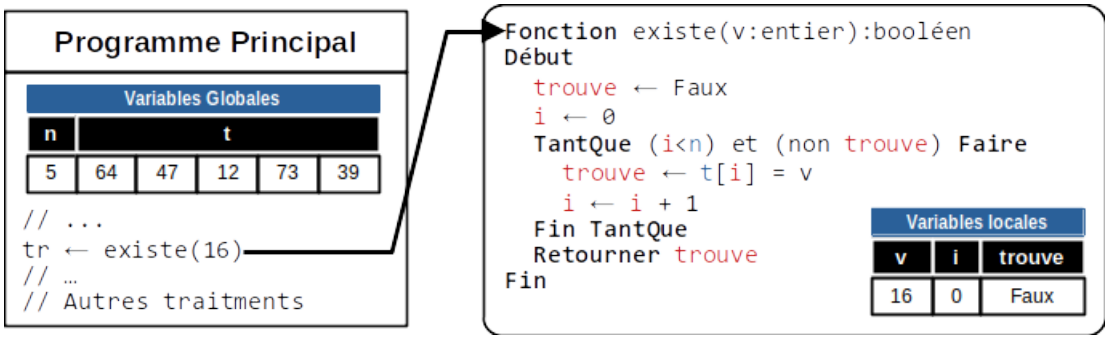

Figure 10, Variables globales et Variables locales

L'exemple précédent est **à éviter** car il contre-dit l'esprit de la modularité. **Il est conseillé vivement d'éviter les variables globales**.

```
// A éviter
// Dépend des variables du PP
Fonction existe(v:entier):booléen
```

```
// Meilleure écriture
// Indépendant du PP
Fonction existe(v:entier;t:tab;n:entier):booléen
```

Applications

Nombre heureux

Un nombre heureux est un entier strictement positif, qui lorsqu'on additionne les carrés de chacun de ses chiffres, puis on additionne les carrés des chiffres de la somme obtenue et ainsi de suite, on obtient un entier à un seul chiffre égal à 9.
Ecrire l'algorithme d'un programme qui permet de chercher et afficher tous les nombres heureux de `p` (2 ≤ p ≤ 5) chiffres.

Exemples

```
Exemple 1
Nombre de chiffres ? 3
122 est heureux.
125 est heureux.
...
954 est heureux.
28 nombres heureux dans l'intervalle [100, 999].
```

```
Exemple 2
Nombre de chiffres ? 4
1022 est heureux.
1025 est heureux.
...
9985 est heureux.
274 nombres heureux dans l'intervalle [1000, 9999].
```

Nombres frères

Deux entiers `n1` et `n2` sont dits frères ssi chaque chiffre de `n1` apparaît au moins une fois dans `n2` et inversement (avec n1 > 0 et n2 > 0).
Ecrire l'algorithme d'un programme qui saisit deux entiers `n1` et `n2` puis vérifie s'ils sont frères ou non.

Exemples

```
Exemple 1
Donner n1 > 0 ? 1234
Donner n2 > 0 ? 434212
1234 et 434212 sont frères
```

```
Exemple 2
Donner n1 > 0 ? 154
Donner n2 > 0 ? 341
154 et 341 ne sont pas frères
```

Nombres super premiers

Un nombre est dit super premier ssi il est premier et si en supprimant des chiffres à partir de sa droite, le nombre restant est toujours premier.
Ecrire l'algorithme d'un programme qui cherche tous les nombres super premiers dans un intervalle [a, b] donné (100 < a < b < 100000).

Exemples

```
Exemple 1
a > 100 ? 101
101 < b < 100000 ? 1000
233 est super premier.
239 est super premier.
...
739 est super premier.
797 est super premier.
14 nombres super premiers dans [101, 1000]
```

```
Exemple 2
a > 100 ? 1000
1000 < b < 100000 ? 10000
2333 est super premier.
2339 est super premier.
...
7333 est super premier.
7393 est super premier.
16 nombres super premiers dans [1000, 10000]
```

Billet

Sur les billets d'avion d'une companie aérienne figure un code de 11 lettres alphanumériques (chiffres et lettres majuscules).
Pour vérifier l'authenticité d'un billet, on remplace la lettre du code par son rang alphabétique pour obtenir un nombre de 12 ou de 13 chiffres.
Si le reste de la division par 9 de la somme des chiffres de ce nombre est égale à 8, ce billet est authentique, sinon c'est un faux billet.
Ecrire l'algorithme d'un programme qui permet de vérifier l'authenticité d'un billet à partir de son code.

Exemples

```
Exemple 1
Numéro de billet ? AR798NKV789
Somme de contrôle : 6
Billet non authentique.
```

```
Exemple 2
Numéro de billet ? AR798NKV989
Somme de contrôle : 8
Billet authentique.
```

EAN13

Le code EAN13 (European Article Numbering à 13 chiffres) est un code à barres utilisé sur l'ensemble des produits de grande consommation. Ce code est composé de 13 chiffres.

Figure 11, Exemples de codes à barres

Un code EAN13 est formé par :

- Un identifiant du produit `q` formé par les 12 premiers chiffres à gauche.
- La clé de contrôle `cc` formé par le dernier chiffre à droite.

Pour vérifier qu'un nombre de 13 chiffres est un code EAN13 valide on applique le principe suivant :

1. On calcule la somme `s` des chiffres de `q` en commençant par le chiffre le plus à gauche et en multipliant les chiffres de rang pair par 3. Le rang du premier chiffre le plus à gauche est 1, celui du deuxième chiffre le plus à gauche est 2, etc.
2. On calcule le reste `r` de la division euclidienne de `s` par 10.
3. On calcule `p` qui est égal à `(10-r)`
4. Si `p` est égal à `cc` alors le code est valide.

Ecrire l'algorithme d'un programme qui permet de saisir un nombre `n` de 13 chiffres et vérifie s'il est un code EAN13 ou non.

Exemples

Vérification du code à barre : 1234567890128

q = "123456789012" et cc = "8"

q	1	2	3	4	5	6	7	8	9	0	1	2	8 = 10-2
s	1	2 × 3	3	4 × 3	5	6 × 3	7	8 × 3	9	0 × 3	1	2 × 3	=92

Vérification du code à barre : 9780201379624

q = "978020137962" et cc = "4"

q	9	7	8	0	2	0	1	3	7	9	6	2	4 = 10-6
s	9	7 × 3	8	0 × 3	2	0 × 3	1	3 × 3	7	9 × 3	6	2 × 3	=96

```
Exemple 1
Code EAN13 ? 9780201379624
Code EAN13 valide.
```

```
Exemple 2
Code EAN13 ? 9780201379625
Code EAN13 invalide.
```

Game of Thrones shop

Les statistiques montrent qu'en 2022 environ 2,46 milliards de personnes ont joué à des jeux vidéo sur une console, un ordinateur ou un mobile. Vu le nombre important des "gamers" on trouve des centaines de boutiques en ligne qui vendent des répliques des articles rencontrés dans les jeux vidéos.
Un "gamer" adepte du jeu Games of Thrones se rend sur l'une de ces boutiques, voir figure 1.
Le "gamer" dispose d'un budget limité. Et il veut faire un programme qui :

- Saisit le nombre d'articles n disponibles dans la boutique. (2 ≤ n ≤ 50)
- Remplit trois tableaux :
  - items : contient les noms des articles qui sont des chaînes d'au moins 3 caractères. Les articles doivent être distincts.
  - categories : contient les catégories des articles, les catégories acceptées sont :

Catégorie	"SW"	"AR"	"SH"	"WE"
Signification	Sword (épée)	Armor (Armure)	Shield (Bouclier)	Weapon (Arme)

  - prices : contient les prix des articles (des entiers ≥ 0)
- Saisit son budget (20 ≤ budget ≤ 300)
- Saisit une catégories parmi les catégories mentionnées dans le tableau précédent.
- Affiche la liste des articles de cette catégorie, dont le prix est inférieur ou égal à son budget.

Exemples : Pour **n = 8** et les articles suivants :

items	The Sword of Jon Snow	Sword of Arya Stark	Widow's Wail Sword	Dragon Leather Bracers Set	The Helm of Loras Tyrell	Stark Infantry Shield	Lannister Shield	The Red Viper's Spear
categories	SW	SW	SW	AR	AR	SH	SH	WE
prices	239	179	649	38	249	257	275	299

Exemple 1

```
Your budget? 202
Select a category? SW
Affordable items:
Sword of Arya Stark - Price: 179
```

Exemple 2

```
Your budget? 182
Select a category? SH
Affordable items:
No items
```

Exemple 3

```
Your budget? 54
Select a category? AR
Affordable items:
Dragon Leather Bracers Set - Price:
```

Écrire l'algorithme de ce programme. **Ne pas oublier le TDNT et le TDO.**

Solution non modulaire

On donne la solution non modulaire de ce problème et on demande de le décomposer en modules et d'écrire l'algorithme de chaque module.

```
Algorithme Boutique
Début
  Répéter
    Ecrire("Number of items [2, 50] ? ") ; Lire(n)
  Jusqu'à (2 ≤ n ≤ 50)

  Pour i de 0 à n-1 Faire
    Ecrire("Item n°", i)
    Répéter
      Ecrire("Name ? ") ; Lire(items[i])
      existe ← Faux
      j ← 0
      TantQue (non existe) et (j < i) Faire
        existe ← items[j] = items[i]
        j ← j + 1
      Fin TantQue
    Jusqu'à (Long(items[i]) ≥ 3) et (non existe)

    Répéter
      Ecrire("Category ? ") ; Lire(categories[i])
    Jusqu'à categories[i] ∈ ["SW", "AR", "SH", "WE"]

    Répéter
      Ecrire("Price ? ") ; Lire(prices[i])
    Jusqu'à prices[i] ≥ 0
  Fin Pour

  Répéter
    Ecrire("Your budget ? ") ; Lire(budget)
  Jusqu'à 20 ≤ budget ≤ 300

  Répéter
    Ecrire("Select a category ? ") ; Lire(catsel)
  Jusqu'à catsel ∈ ["SW", "AR", "SH", "WE"]

  ic ← 0
  Ecrire("Affordable items:")
  Pour i de 0 à n-1 Faire
    Si (categories[i] = catsel) et (budget ≥ prices[i]) Alors
      Ecrire("Item:", items[i], " - Price:", prices[i])
      ic ← ic + 1
    Fin Si
  Fin Pour

  Si ic = 0 Alors
    Ecrire("N items.")
  Fin Si
Fin
```

TDNT
tabe = tableau de 50 entier
tabch = tableau de 50 chaîne

TDO

Objet	Type/Nature
n, i, j, ic, budget	entier
catsel	chaîne
existe	booléen
items, categories	tabch
prices	tabe